

INTERNSHIP PROJECT REPORT

Task 2

Database Integration & Flyway Migrations

using Spring Boot, Spring Data JPA, MySQL, and Flyway

Submitted By

Swapnil Soni

College

VIT Bhopal University

Branch

B.Tech Computer Science Engineering

Program

InternSpark AICTE-listed Internship Program

Table of Contents

Table of Contents.....	2
1. Introduction	3
2. Objective	4
3. Technologies Used	5
4. Project Structure	6
4.1 Database Configuration.....	6
5. Methodology	7
6. Flyway Migration Workflow.....	8
7. REST API Endpoint.....	9
8. API Testing	10
9. Key Commands Used	14
9.1 Run Application	14
9.2 Git & Version Control Commands.....	14
9.3 GitHub Repository.....	14
10. Learning Outcomes.....	15
11. Conclusion	16

1. Introduction

This report documents the work completed as part of Task 2 of the InternSpark AICTE-listed Internship Program. The task involved building a database integration project using Java Spring Boot, Spring Data JPA, MySQL, and Flyway Migration — a widely adopted tool for managing database schema versioning in enterprise applications.

Flyway is an open-source database migration tool that enforces a version-controlled, repeatable approach to schema management. By applying migration scripts in a defined sequence, Flyway ensures that the database schema evolves consistently across all environments — development, testing, and production.

This project demonstrates the practical integration of Flyway with a Spring Boot backend connected to a MySQL database. The migration scripts handle the creation of the students table, insertion of sample data, and addition of a new column — simulating a realistic schema evolution lifecycle. A REST API endpoint was also implemented and tested using Postman to validate that the migrated schema and seeded data function correctly at the application layer.

2. Objective

The primary objectives of this project are to:

- Demonstrate database integration using Spring Boot and MySQL, following production-ready configuration practices.
- Implement Flyway Migration to manage database schema versioning through sequential, versioned SQL scripts.
- Design and execute three distinct migration scripts that reflect a realistic schema evolution workflow:
 - V1 — Initial table creation.
 - V2 — Sample data seeding.
 - V3 — Schema alteration via addition of a new column.
- Expose a REST API endpoint (GET /students) and validate its response against the migrated and seeded database.
- Follow clean project structure and layered Spring Boot architecture best practices.
- Apply version control discipline using Git and publish the project on GitHub.

3. Technologies Used

Technology	Purpose
Java 17	Core programming language for backend business logic and API implementation.
Spring Boot	Framework for building and running production-ready REST applications rapidly.
Spring Data JPA	ORM layer for abstracting database interactions using repository interfaces.
MySQL	Relational database management system for persistent schema and data storage.
Flyway Migration	Database versioning tool for applying and tracking schema migration scripts.
Lombok	Reduces Java boilerplate by auto-generating getters, setters, constructors, and builders.
Maven	Dependency management and project build automation tool.
Postman	API client used for testing REST endpoints and verifying response payloads.
Git & GitHub	Version control system and remote repository hosting for collaborative development.
IntelliJ IDEA	Integrated Development Environment used throughout the project lifecycle.

4. Project Structure

The project follows the standard Spring Boot layered architecture, maintaining clear separation of concerns across distinct packages. Flyway migration scripts are placed in the standard resources/db/migration directory, which Flyway automatically scans on application startup.

```
flyway-migration-demo/
├── controller/      - Exposes the REST API endpoint (/students).
├── entity/          - Defines the Student JPA entity mapped to the students
table.
├── repository/      - Extends JpaRepository for built-in CRUD database
operations.
├── service/         - Contains business logic bridging the controller and
repository layers.
├── resources/
│   ├── application.properties - Spring Boot & Flyway configuration settings.
│   └── db/migration/
│       ├── V1__create_students_table.sql
│       ├── V2__insert_sample_data.sql
│       └── V3__add_phone_column.sql
```

4.1 Database Configuration

Property	Value
Database Name	migration_db
Table Name	students
Migration Path	classpath:db/migration
Flyway Baseline	Enabled via Spring Boot auto-configuration
DDL Auto	validate (Flyway manages schema; Hibernate validates only)

5. Methodology

The project was developed following a structured, step-by-step backend development workflow aligned with industry practices for database migration management:

Step 1: Project Initialization

Created a new Spring Boot project via Spring Initializr with the following dependencies: Spring Web, Spring Data JPA, MySQL Driver, Flyway Migration, and Lombok. Maven was selected as the build tool.

Step 2: Entity Design

Defined the Student entity class with fields: id, name, email, course, and phone. The class was annotated with `@Entity`, `@Table(name="students")`, and Lombok annotations (`@Data`, `@NoArgsConstructor`, `@AllArgsConstructor`) to reduce boilerplate.

Step 3: Repository & Service Layers

Created `StudentRepository` extending `JpaRepository<Student, Long>` to inherit CRUD methods. Implemented `StudentService` containing the business logic for retrieving all student records.

Step 4: Controller Layer

Built `StudentController` using `@RestController` and `@RequestMapping("/students")`, exposing a GET endpoint that returns the full list of students as a JSON array.

Step 5: Database & Flyway Configuration

Configured `application.properties` with the MySQL JDBC connection URL, credentials, and Flyway settings. Set `spring.jpa.hibernate.ddl-auto=validate` to allow Flyway to own schema management entirely.

Step 6: Migration Script Authoring

Authored three versioned SQL scripts placed under `resources/db/migration`. Each file follows Flyway's mandatory naming convention: `V{version}__{description}.sql`. These scripts were applied automatically on application startup in sequential order.

Step 7: Application Testing

Ran the application using `mvn spring-boot:run`. Verified that Flyway applied all three migration scripts successfully by observing the startup logs and inspecting the `flyway_schema_history` table in MySQL Workbench. Tested the GET `/students` endpoint in Postman.

Step 8: Version Control

Initialized a local Git repository, committed the code with descriptive commit messages, and pushed the complete project to GitHub for remote version control and reference.

6. Flyway Migration Workflow

Flyway applies migration scripts automatically during application startup by scanning the configured migration path. Each script is executed exactly once and its status is recorded in the `flyway_schema_history` table within the same database. The three scripts used in this project simulate a progressive schema evolution workflow.

V1__create_students_table.sql

Creates the initial students table with the primary key and core student attributes.

```
CREATE TABLE students (  
  id      BIGINT      NOT NULL AUTO_INCREMENT,  
  name    VARCHAR(100) NOT NULL,  
  email   VARCHAR(150) NOT NULL UNIQUE,  
  course  VARCHAR(100) NOT NULL,  
  PRIMARY KEY (id)  
);
```

V2__insert_sample_data.sql

Seeds the students table with three sample records to validate data persistence and API retrieval.

```
INSERT INTO students (name, email, course) VALUES  
  ('Swapnil Soni', 'swapnil@vitbhopal.ac.in', 'Computer Science Engineering'),  
  ('Riya Sharma', 'riya@vitbhopal.ac.in', 'Electronics Engineering'),  
  ('Arjun Mehta', 'arjun@vitbhopal.ac.in', 'Information Technology');
```

V3__add_phone_column.sql

Adds a phone column to the existing students table, simulating a real-world schema alteration after initial deployment.

```
ALTER TABLE students  
  ADD COLUMN phone VARCHAR(15) DEFAULT NULL;
```

On application startup, Flyway detects the pending migration scripts, executes them in version order (V1 → V2 → V3), and marks each as successful in the `flyway_schema_history` table. If the application is restarted, Flyway identifies that all scripts have already been applied and skips them, ensuring idempotent schema management.

7. REST API Endpoint

The following REST API endpoint was implemented and tested as part of this task. The endpoint retrieves all student records from the MySQL database, including the phone column added by the V3 migration script.

Method	Endpoint	Description	Parameters
GET	/students	Retrieves a complete list of all student records from the database.	None

Sample JSON Response:

```
[
  {
    "id": 1,
    "name": "Swapnil Soni",
    "email": "swapnil@vitbhopal.ac.in",
    "course": "Computer Science Engineering",
    "phone": null
  },
  {
    "id": 2,
    "name": "Riya Sharma",
    "email": "riya@vitbhopal.ac.in",
    "course": "Electronics Engineering",
    "phone": null
  },
  {
    "id": 3,
    "name": "Arjun Mehta",
    "email": "arjun@vitbhopal.ac.in",
    "course": "Information Technology",
    "phone": null
  }
]
```

8. API Testing

All components of the project were verified through a combination of application startup log inspection and Postman API testing. Each step confirms that Flyway migrations applied correctly and that the REST endpoint retrieves accurate, persisted data from the MySQL database.

Figure 1: Project Structure

Displays the complete Maven project layout in IntelliJ IDEA, including the db/migration folder containing all three Flyway scripts.

[Screenshot — IntelliJ IDEA project tree showing controller, entity, repository, service, resources, and db/migration packages]

Figure 2: application.properties Configuration

Shows the Spring Boot application.properties file configured with MySQL datasource URL, credentials, Flyway settings, and JPA validation mode.

```
# MySQL Datasource
spring.datasource.url=jdbc:mysql://localhost:3306/migration_db
spring.datasource.username=root
spring.datasource.password=yourpassword
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Flyway
spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration
```

Figure 3: Flyway Migration Startup Logs

Displays the IntelliJ IDEA console output confirming that all three Flyway migration scripts (V1, V2, V3) were detected and applied successfully on application startup.

```
INFO --- FlywayAutoConfiguration : Flyway Community Edition
INFO --- DbMigrate : Current version of schema: << Empty Schema >>
INFO --- DbMigrate : Migrating schema to version 1 - create students table
INFO --- DbMigrate : Migrating schema to version 2 - insert sample data
INFO --- DbMigrate : Migrating schema to version 3 - add phone column
INFO --- DbMigrate : Successfully applied 3 migrations to schema (execution time
00:00.415s)
```

Figure 4: flyway_schema_history Table in MySQL Workbench

Shows the `flyway_schema_history` table with three records confirming successful execution of V1, V2, and V3 migration scripts, along with their checksums, execution times, and success status.

installed_rank	version	description	type	script	success
1	1	create students table	SQL	V1__create_students_table.sql	1
2	2	insert sample data	SQL	V2__insert_sample_data.sql	1
3	3	add phone column	SQL	V3__add_phone_column.sql	1

Figure 5: students Table in MySQL Workbench

Shows the `students` table with the three seeded records inserted by V2 migration, and the phone column added by V3 migration, all persisted correctly in the `migration_db` database.

id	name	email	course	phone
1	Swapnil Soni	swapnil@vitbhopal.ac.in	Computer Science Engg.	NULL
2	Riya Sharma	riya@vitbhopal.ac.in	Electronics Engineering	NULL
3	Arjun Mehta	arjun@vitbhopal.ac.in	Information Technology	NULL

Figure 6: GET /students — Postman API Request

Demonstrates a successful GET request to `http://localhost:8080/students`, returning a 200 OK status and a JSON array containing all three student records persisted by the Flyway migration scripts.

```
GET http://localhost:8080/students
```

```
Response: 200 OK
```

```
[
  { "id": 1, "name": "Swapnil Soni", "email": "swapnil@vitbhopal.ac.in",
    "course": "Computer Science Engineering", "phone": null },
  { "id": 2, "name": "Riya Sharma", "email": "riya@vitbhopal.ac.in",
    "course": "Electronics Engineering", "phone": null },
  { "id": 3, "name": "Arjun Mehta", "email": "arjun@vitbhopal.ac.in",
    "course": "Information Technology", "phone": null }
]
```

9. Key Commands Used

9.1 Run Application

```
mvn spring-boot:run
```

9.2 Git & Version Control Commands

```
git init                                # Initialise a new local Git
repository.
git add .                               # Stage all project files
for commit.
git commit -m "Added Task 2 - Database Integration & Flyway Migrations"
                                         # Commit with a descriptive
message.
git push origin main                   # Push committed code to
remote GitHub.
```

9.3 GitHub Repository

Repository: <https://github.com/Swapnil-code-maker/InternSpark-Internship>

10. Learning Outcomes

Completion of this task resulted in the following key learnings:

Understood the purpose and mechanics of Flyway Migration.

Learned how Flyway automatically discovers, versions, and applies SQL migration scripts in order, and how the flyway_schema_history table records the execution state of each script.

Practiced SQL-based schema versioning.

Gained experience writing migration scripts following the V{version}__{description}.sql naming convention, covering table creation, data seeding, and schema alteration across three script versions.

Deepened understanding of Spring Boot and Spring Data JPA integration.

Configured spring.jpa.hibernate.ddl-auto=validate to delegate schema ownership to Flyway, which is the recommended approach in production-grade Spring Boot applications.

Applied Lombok to reduce boilerplate code.

Used Lombok annotations (@Data, @NoArgsConstructor, @AllArgsConstructor) in the Student entity to automatically generate common Java methods without manual implementation.

Reinforced REST API testing with Postman.

Tested the GET /students endpoint, verified the 200 OK response status, and confirmed that the JSON payload accurately reflected the migrated schema and seeded data.

Maintained version control discipline using Git and GitHub.

Continued applying professional Git practices by committing with descriptive messages and pushing the full project to the remote repository for reference and evaluation.

11. Conclusion

This project successfully demonstrated database integration using Spring Boot, Spring Data JPA, MySQL, and Flyway Migration. All three migration scripts were applied in the correct sequence, the students table was created and populated with sample records, and the phone column was added as part of a schema alteration migration — all executed automatically by Flyway on application startup.

The REST API endpoint GET /students was tested and confirmed to return the full list of student records as a JSON array with a 200 OK response, validating end-to-end database connectivity, schema correctness, and application-layer data retrieval.

This task provided practical exposure to an essential skill in backend and DevOps engineering: managing database schema changes in a controlled, traceable, and repeatable manner. The knowledge gained through working with Flyway Migration, combined with the continued practice of Spring Boot layered architecture and REST API development, builds a strong foundation for the more complex tasks ahead in the internship program.

Submitted By

Swapnil Soni

B.Tech CSE — VIT Bhopal University